

第三章 RISC-V汇编及其指令系统

May 12th, By Rafael

RISC-V汇编语言

- 通用指令格式

op, dst, src1, src2

operation: 操作名字
destination: 目标寄存器
source 1&2: 第一和第二个源操作数寄存器

寄存器	助记符	备注
x0	zero	固定值为0
x1	ra	返回地址(Return Address)
x2	sp	栈指针(Stack Pointer)
x3	gp	全局指针(Global Pointer)
x4	tp	线程指针(Thread Pointer)
x5-x7	t0-t2	临时寄存器（temporary register）
x8	s0/fp	save寄存器/帧指针(Frame Pointer)
x9	s1	save寄存器（Save Register）
x10-x11	a0-a1	函数参数 / 函数返回值(Return Value)
x12-x17	a2-a7	函数参数(Function Argument)
x18-x27	s2-s11	save寄存器
x28-x31	t3-6	临时寄存器

函数调用中保留的寄存器 sp, gp, tp, x8-x9,x18-x27

- 字节排布

大端机：最高的字节在最低的地址，字的地址等于最高字节的地址
小端机：最低的字节在最低的地址，字的地址等于最低字节的地址

- 栈的使用
 - RISC-V汇编子程序使用寄存器（全局变量）
 - 属于现场的寄存器：*t0 – t6, a0 – a7, s0 – s11, ra, sp*

- 在调用函数之前需要一个保存旧值的地方，在返回时还原它们，然后删除。在RISC-V中使用栈来完成这个功能
- 按历史惯例，RISC-V中栈按照从高到低的地址顺序增长（即栈底位于高地址，栈顶位于低地址）
- 栈中储存的数据
 - 需要保存的参数/save寄存器的值
 - 函数执行完成后，返回下一条“指令”的地址
 - 其他局部变量（例如函数中使用的数组或结构体）的空间

RISC-V指令表示

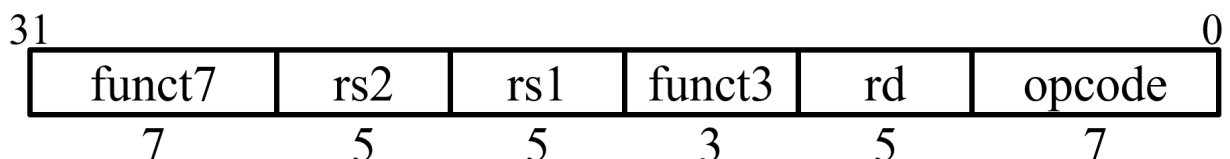
- RISC-V定义了六种基本的指令格式

R	用于寄存器之间的操作
I	用于短立即数和取数 load 操作
S	用于存数 store 操作
B	用于条件跳转操作
U	用于长立即数操作
J	用于无条件跳转操作

R 型	funct7	rs2	rs1	funct3	rd	opcode
I 型	imm[11:0]		rs1	funct3	rd	opcode
S 型	Imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
B 型	Imm[12,10:5]	rs2	rs1	funct3	imm[4:1,11]	opcode
J 型	Imm[20,10:1,11,19:12]				rd	opcode
U 型	Imm[31:12]				rd	opcode

考试给这个表格和位数以及各种指令的效果，如add--R--rd=rs1+rs2

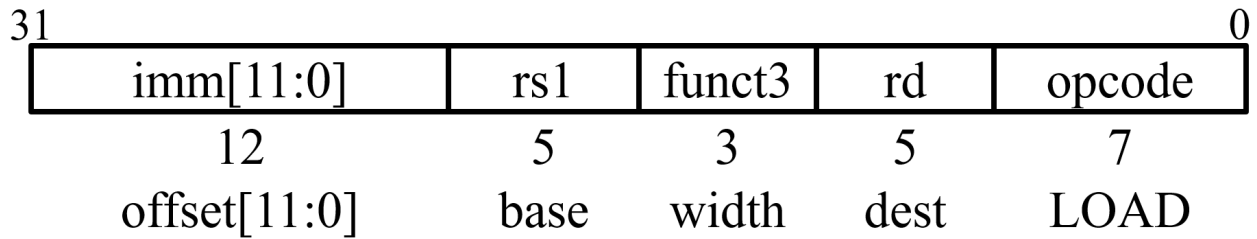
- R型指令



- opcode: 指定它是什么指令
- funct7+funct3: 这两个字段结合操作码描述要执行的操作
- rs1&rs2: 指定第一/二个寄存器操作数
- rd: 指定接受计算结果的寄存器
- 每个字段保存一个5位无符号整数 (0-31), 对应于一个寄存器号 (x0-x31)

```
add x5, x6, x7 # x5 = x6 + x7
```

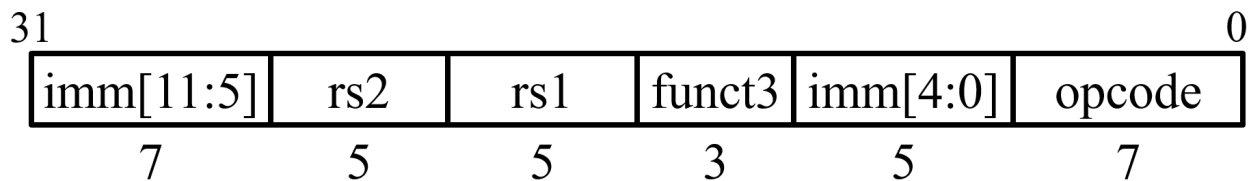
• I型指令



- imm[11: 0]: 有符号立即数(常数), 若大于12位则在算术运算前符号扩展到32位

```
lui x5, 0x12345 # 把 0x12345000 放入 x5 (加载高 20 位)
addi x5, x5, 0x678 # 加上低 12 位 0x678 → 得到 0x12345678
```

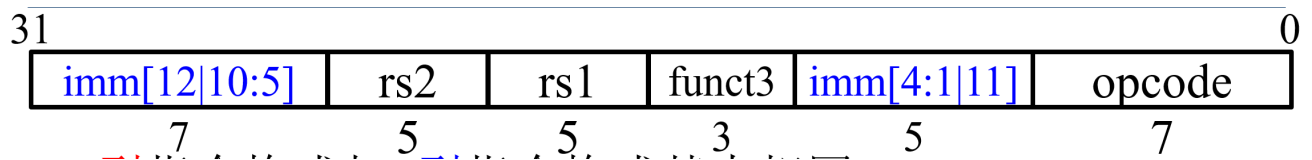
• S型指令



- rs1为要写入的内存地址, rs2为要写入的数据, imm是偏移量
- 12位immediate是分开的, 保证rs2位置不变
- 值不写入寄存器, 所以没有rd
- 将低位的5位立即数移到其他指令中的rd字段所在的位置——保持rs1/rs2字段在同一位置

```
sw x5, 8(x6) # 将寄存器x5的数据储存到地址x6+8所指的位置
```

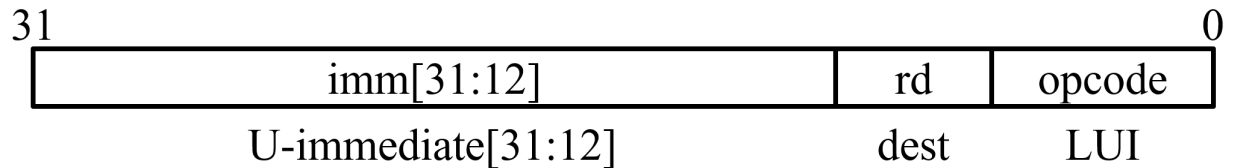
• B型指令



- 将imm作为相对PC(Program Counter)的补码偏移量 (PC相对寻址)，跳转地址的实际偏移是立即数字段的值乘以 2
- 12位的imm表示13位有符号字节地址，最低位有不储存的0
- 将偏移量视为以字为单位，在相对寻址前，先将偏移量乘以4,得到字节偏移量，再加上PC中的基地址
- 为了实现对可能的扩展指令的支持，在实际的RISC-V指令设计中，分支转移指令的偏移量都是以2字节为单位
- RISC-V条件分支指令实际上只能访问相对PC地址前后 $2^{12} \times 16$ 位范围内的指令
- 基于RISC-V的扩展指令集支持16位压缩编码指令，也支持16位长度倍数的可变长度指令

```
beq x5, x6, label # 若 x5 == x6, 则跳转
```

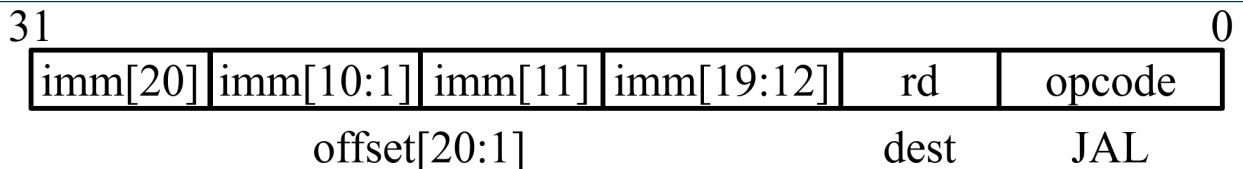
• U型指令



- 长立即数操作：高20位立即数字段和5位目的寄存器字段
- 上面的lui就是U型指令,将长立即数写入rd高20位并清除低12位
- ADDI 立即数总是进行符号扩展，如果高位为1，将从高位的20位中减去1

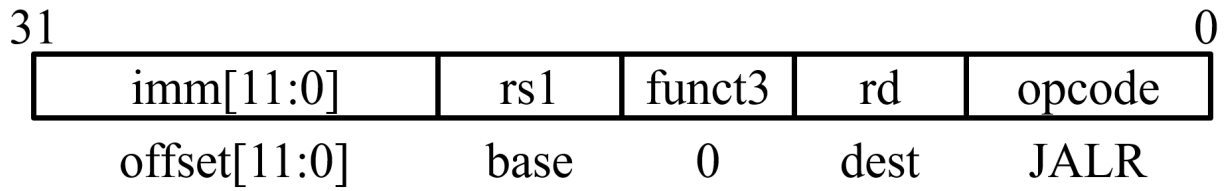
```
lui x10, 0xDEADC # x10 = 0xDEADC000
addi x10, x10, -0x111 (0xFFFFFEEF) # x10 = 0xDEADBEEF
```

• J型指令



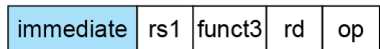
- JAL指令：跳转到PC+offset的位置，将PC+4(跳转后下一条指令)写入rd中
jal ra, label # 跳转到label, 返回地址(PC+4)存入ra
- $PC = PC + offset$
- 访问相对PC,以2字节为单位的 $\pm 2^{19}$ 范围内的地址空间

- jalr指令

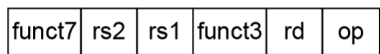


- 属于I型指令
- 将PC+4写入rd, 设置PC = rs1 + imm
- 立即数不×2字节
- 寻址模式

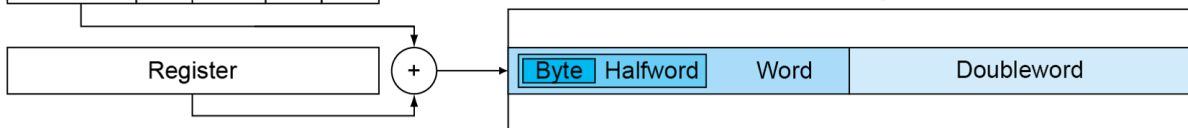
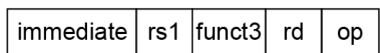
1. Immediate addressing



2. Register addressing



3. Base addressing



4. PC-relative addressing

